

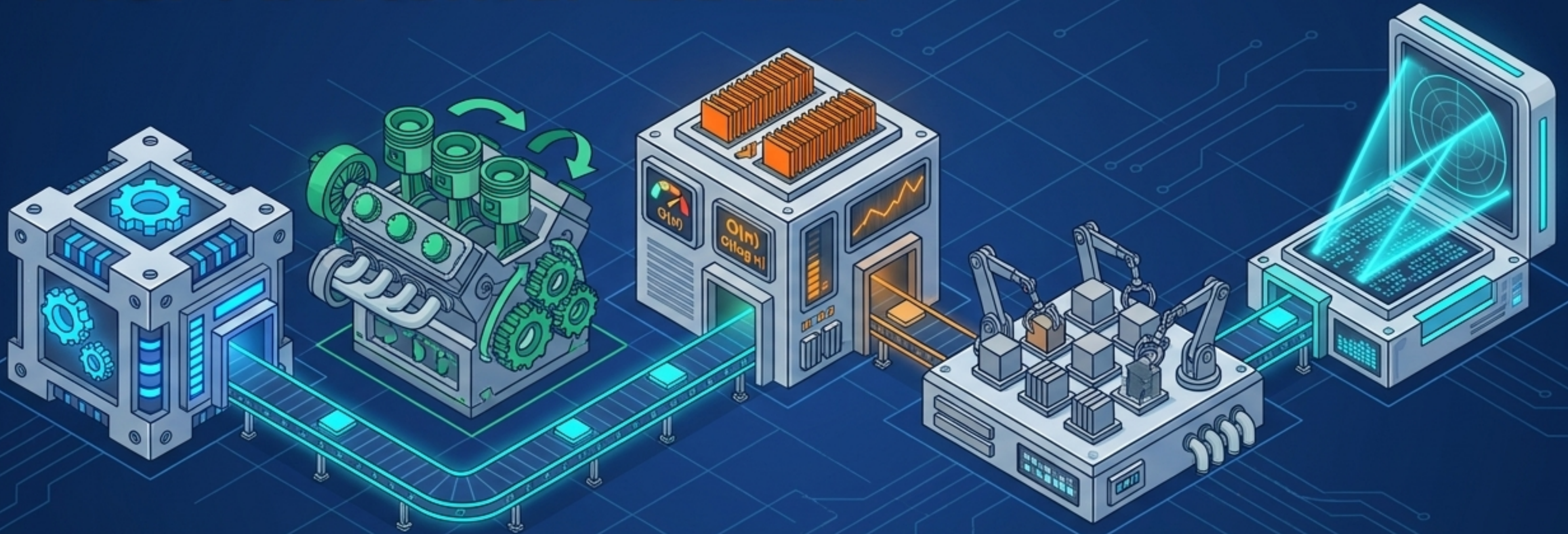
Algoritma dan Pemrograman 2: Blueprint Integrasi Sistem

Sintesis Komprehensif Arsitektur Perangkat Lunak (Merangkum Pertemuan 1 – 7)



Tim Pengampu Algoritma & Pemrograman | Persiapan Ujian Tengah Semester (UTS)

The Builder's Toolkit: 5 Pilar Arsitektur Sistem



Pilar 1: Struktur (IPO & Modular)

- Desain pondasi dan dekomposisi masalah.
Algorithmic Terms,
 $O(\log n)$, $O(\log n)$, ...

Pilar 2: Mesin (Iterasi & Rekursi)

- Mesin penggerak repetisi dan pemecahan dinamis.

Pilar 3: Skala (Analisis Big-O)

- Evaluasi efisiensi dan ketahanan sistem.

Pilar 4: Organisasi (Sorting)

- Menjinakkan kekacauan data menjadi keteraturan.

Pilar 5: Navigasi (Searching)

- Pencarian presisi tinggi dalam dimensi data.

Kerangka Berpikir: Transformasi IPO (Input - Process - Output)



IPO mengubah masalah abstrak menjadi langkah konkret yang dapat dieksekusi.

Strategi Dekomposisi: Mengisolasi Kompleksitas

KOMPLEKSITAS
MONOLITIK



MODUL INDEPENDEN
& TERUKUR



Single Responsibility

Satu fungsi = Tepat satu tugas logis.



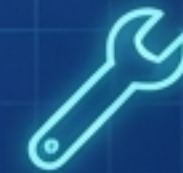
Reusability

Dapat digunakan ulang di proyek atau bagian lain.



Readability

Struktur program jauh lebih mudah dibaca manusia.



Maintainability

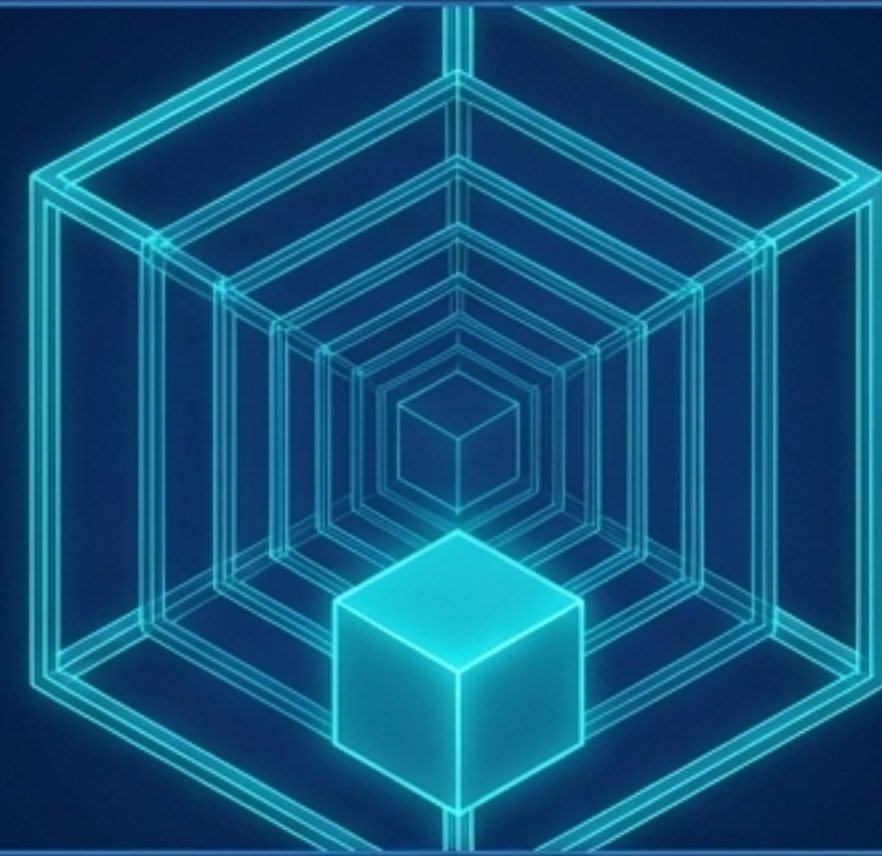
Bug mudah dilacak, fitur baru mudah ditambah tanpa merusak sistem.

Mesin Penggerak: Rekursi vs Iterasi



Iterasi (Mesin Loop)

Menggunakan struktur perulangan kontrol (**for/while**) untuk memperbarui variabel lokal.



Rekursi (Fractal Penyelaman)

Sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan versi yang lebih kecil dari masalah aslinya.



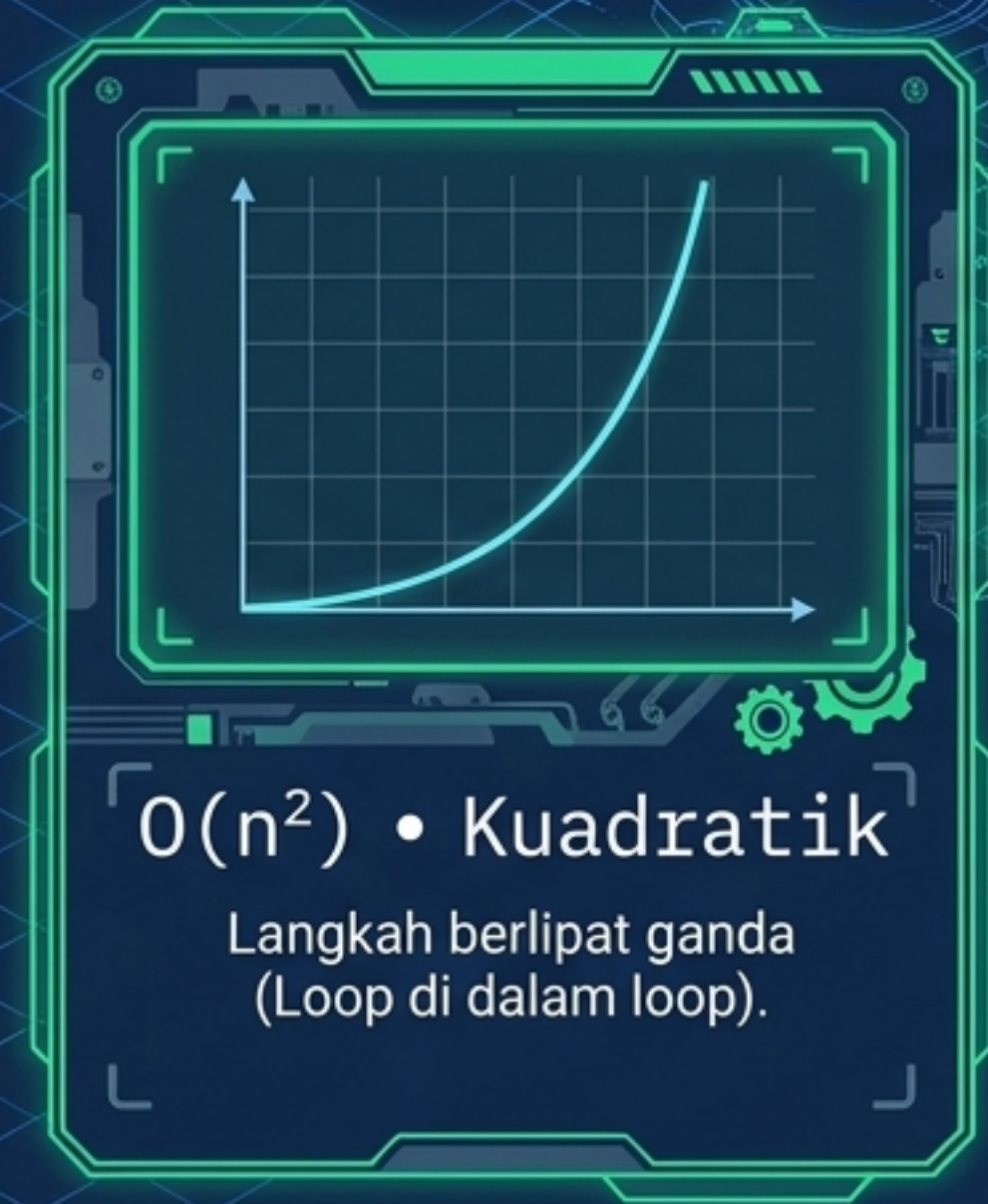
Anatomi Wajib Rekursi

Rekursi wajib memiliki **Base Case** (Kondisi Berhenti / Titik Jangkar). Tanpa Base Case, akan terjadi **Infinite Recursion** yang menyebabkan **Call Stack** meluap dan program **crash**!

Matriks Sintesis: Kapan Menggunakan Rekursi vs Iterasi?

	Rekursi	Iterasi
Mekanisme Utama	Memanggil fungsi dari dalam fungsi (<code>Function Call</code>).	Menggunakan struktur perulangan (<code>Looping: for/while</code>).
Efisiensi Memori	Lebih boros (Membangun <code>Call Stack</code> berlapis di memori untuk setiap panggilan).	Lebih efisien (Hanya memperbarui variabel lokal yang sama).
Gaya & Karakteristik Kode	Elegan, ringkas, ideal untuk struktur pohon/data bercabang tinggi (seperti Algoritma <code>Divide and Conquer</code>).	Cenderung lebih panjang secara prosedural, ideal untuk penelusuran data linier tunggal.

Analisis Big-O: Membaca Bahasa Skala



Mengapa Konstanta Memudar?

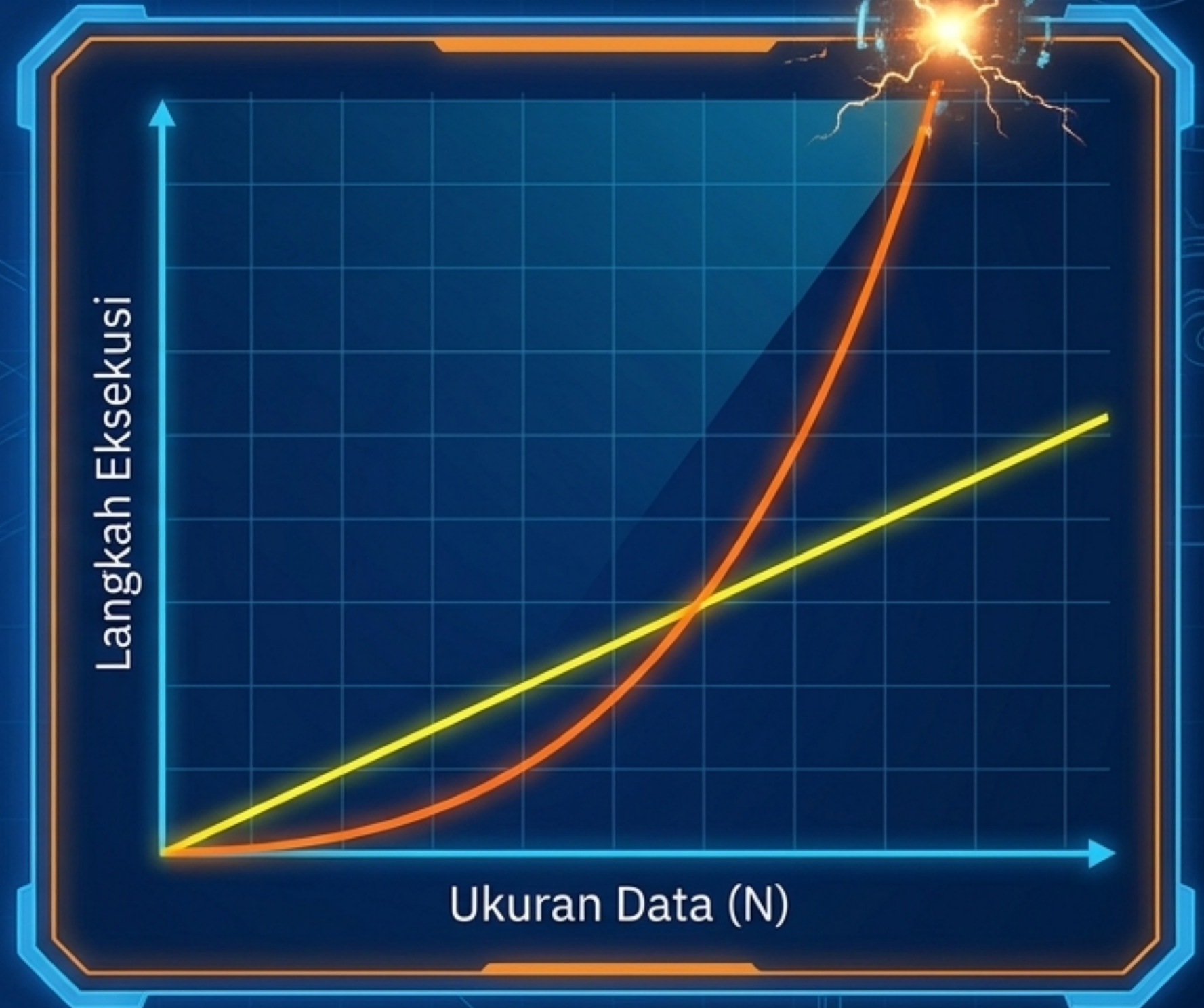
Dalam Big-O, kita mencari pola dominan saat data mendekati tak terhingga. Formula $n + n = 2n$ akan disederhanakan murni menjadi $O(n)$. Konstanta pengali diabaikan.

$$n + n = \cancel{2n} \rightarrow O(n)$$

Ledakan Skala: Bahaya Tersembunyi $O(n^2)$

Ukuran Data (N)	$O(n)$	$O(n^2)$
N = 10	10	100
N = 100	100	10,000
N = 1,000	1,000	1,000,000

⚠ Pada skala data masif, sebuah algoritma kuadratik $O(n^2)$ akan runtuh dan menyebabkan ledakan iterasi.



Algoritma Pencarian: Linear vs Binary Search

Linear Search $O(n)$ – Menyisir Manual



Iterasi tanpa henti.
Memeriksa elemen satu per satu.



Pada $N=1.000$, butuh maksimal
1.000 langkah komputasi.

Binary Search $O(\log n)$ – Membelah Dimensi



Mengeliminasi 50% ruang data yang
tidak relevan di setiap iterasi.

Pada $N=1.000$, butuh maksimal
HANYA 10 langkah!



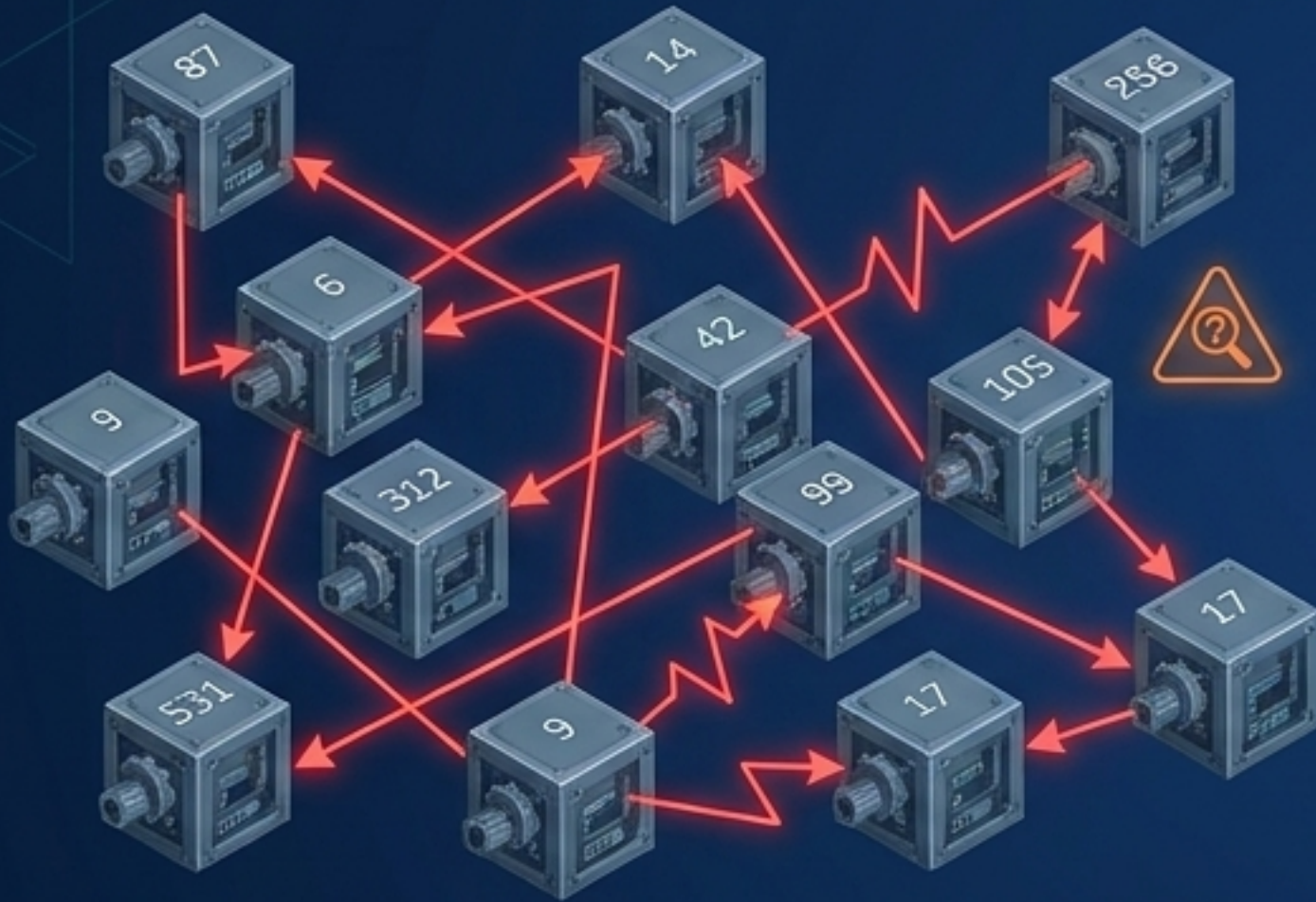
Prasyarat Mutlak:
Data **WAJIB** dalam
keadaan terurut.

Kerangka Keputusan Arsitektur: Memilih Senjata Pencarian



Esensi Sorting: Menjinakkan Kekacauan Data

Kekacauan



$O(n)$ Efficiency: Poor. Mengharuskan sistem memeriksa data secara buta (Linear).

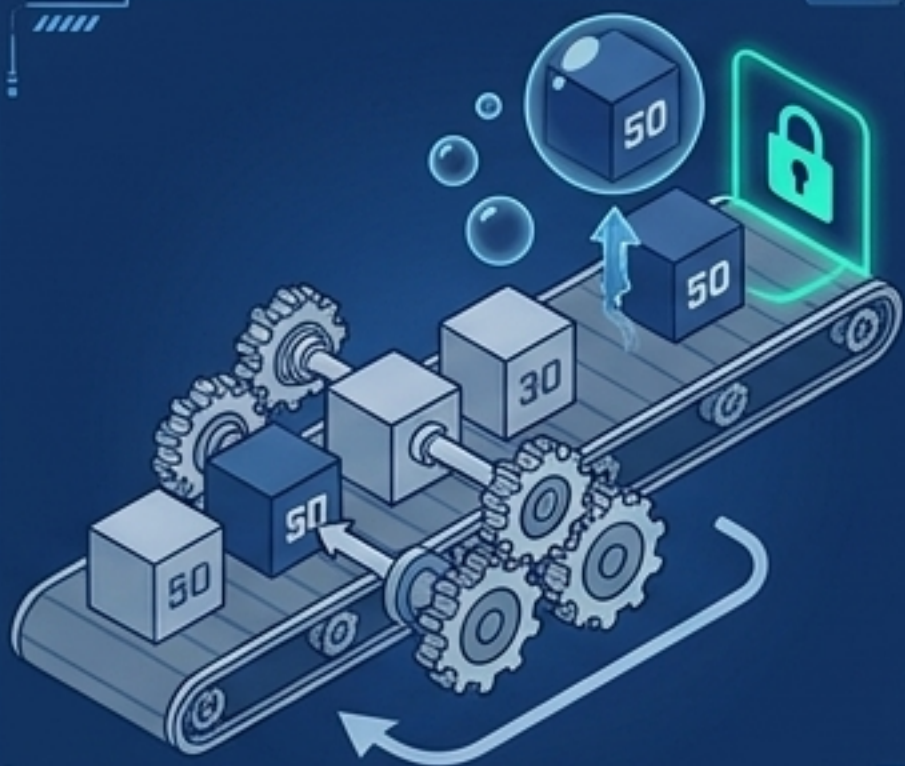
Keteraturan



Prasyarat Sistem: Membuka jalan untuk efisiensi tingkat lanjut (Advanced Prerequisite).

Mekanika 3 Sorting Dasar

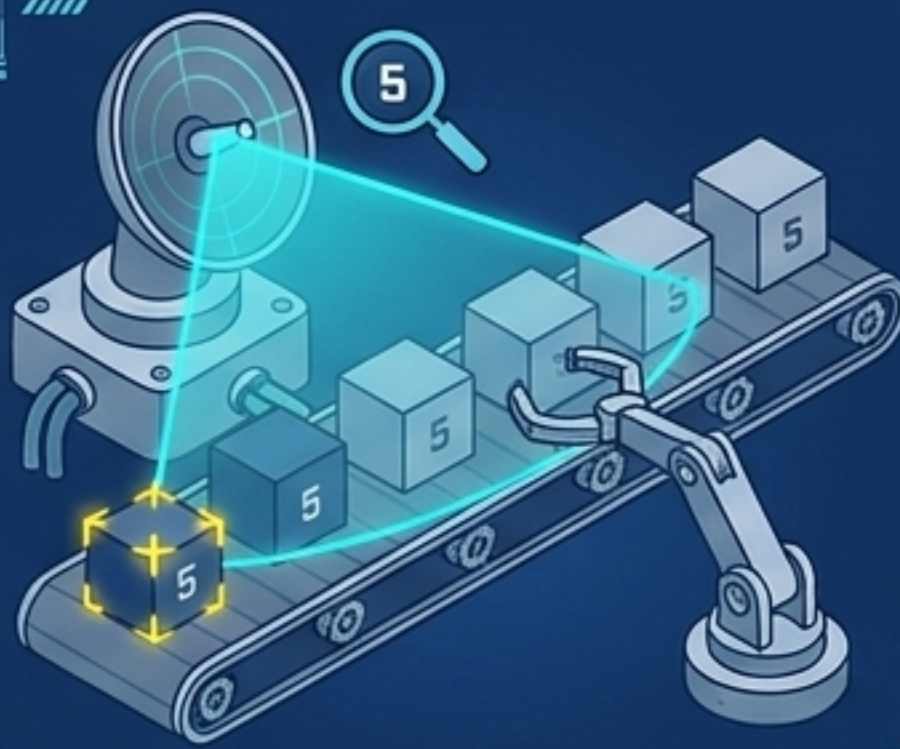
Bubble Sort



Metaphor: Gelembung udara terbesar naik ke permukaan.

Action: Membandingkan dan menggeser/menukar dua elemen bersebelahan secara berulang hingga elemen terbesar terkunci di ujung.

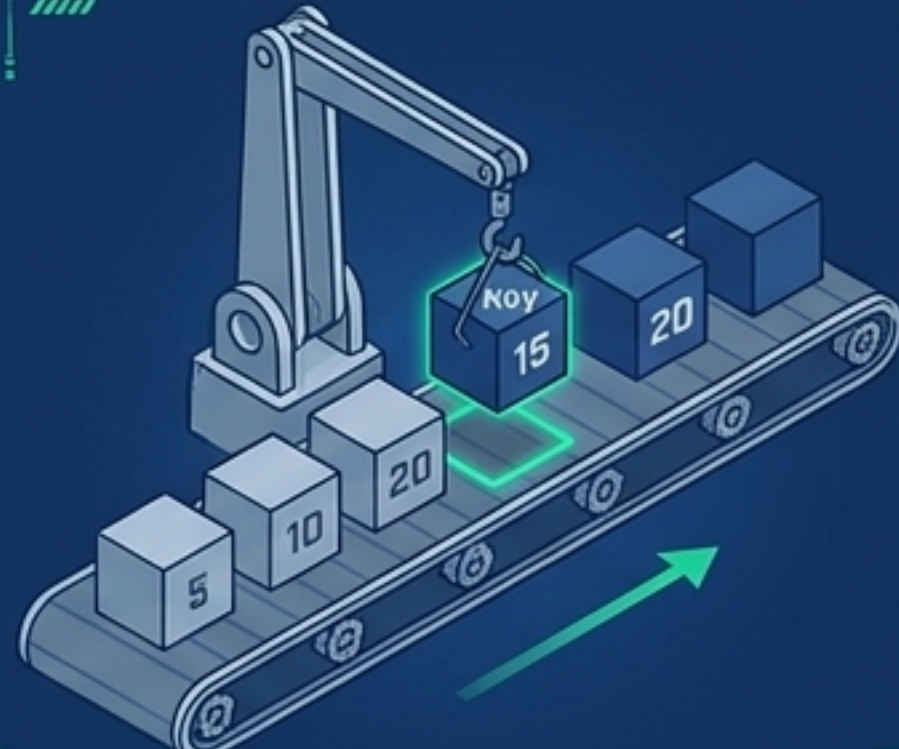
Selection Sort



Metaphor: Sapuan radar mencari target terkecil.

Action: Memindai seluruh area yang belum terurut untuk mencari nilai minimum global, lalu menukarnya ke garis depan.

Insertion Sort



Metaphor: Menyusun kartu remi di tangan.

Action: Mengambil satu elemen (Key) dan menyisipkannya ke ruang kosong yang tepat di area terurut sebelah kiri.

Matriks Diagnostik: Paradoks $O(n^2)$

Ketiganya berkecepatan $O(n^2)$, lalu bagaimana kita memilihnya?

Algoritma	Intensitas Pertukaran (Swaps)	Lingkungan Optimal
Bubble Sort	 SANGAT TINGGI (High CPU writes)	Implementasi sangat sederhana untuk modul pembelajaran dasar.
Selection Sort	 RENDAH (Maksimal 1 kali tukar per iterasi)	Digunakan saat Memory Write (operasi menulis data) sangat mahal/terbatas.
Insertion Sort	 SEDANG (Sifatnya adaptif)	Sangat cepat dan mendekati $O(n)$ jika data hampir terurut (Nearly Sorted / Streaming Data).

Blueprint Integrasi Sistem: Pipeline Evaluasi Total



Analisis Kompleksitas Sistem Total:
Performa keseluruhan sistem dibatasi oleh mata rantai terlemahnya (Biasanya tahap Sorting).

"Pemrograman modular dan integrasi sistem bukan sekadar tentang memecah kode. Ini tentang mengisolasi kompleksitas."

"Program yang baik tidak hanya bekerja (berjalan). Ia dirancang agar mudah dibaca oleh manusia, dapat diskalakan oleh mesin, dan dipertahankan untuk masa depan."

Selamat Menempuh Ujian Tengah Semester!